6부 전반 — 글자에서 의미로

텍스트 인코딩 · 토큰 · 원-핫에서 임베딩으로 · Word2Vec 미니

앱 페이즈 15~18 (히든 스테이지) · 대상: 고등학생 · 활동 시간 약 90분

학년 반 번호 _____ 이름 _____

STEP 1. 텍스트가 숫자가 되기까지

1-1 "안녕하세요" — 코드포인트 표

아래는 다섯 글자의 코드포인트(10진)와 UTF-8 바이트 수입니다. 표를 보고 빈칸을 채우세요.

글자	안	녕	하	세	요
코드포인트(10진)	50504	45397	54616	49464	50836
UTF-8 바이트 수	3	3	3	3	3

"안녕하세요"는 글자 수 = _____개, 총 UTF-8 바이트 = _____ × 3 = _____ 바이트.

한글 음절은 모두 UTF-8에서 정확히 3바이트라는 사실이 위 표에서 확인된다. 그 이유를 한 줄로:

1-2 한·영·이모지 길이 비교

문자열	글자 수	UTF-8 총 바이트
"AI"	2	
"인공지능"	4	
"🤖🧠"	2	

참고 — 영어 1바이트, 한글 3바이트, 이모지 4바이트로 가정하고 빈칸을 직접 곱셈으로 채우세요.

한글이나 이모지가 영어보다 바이트를 더 많이 쓰는 이유는?

STEP 2. 토큰 — 단어보다 작고 글자보다 큰 조각

2-1 토큰 수 비교 — 결과 표

GPT 계열 토크나이저 기준의 토큰 수입니다. 글자 수와 비교하며 패턴을 발견하세요.

예문	글자 수	토큰 수	글자/토큰 비율
Hello world	11	2	
Tokenization is fun	19	4	
안녕하세요	5	약 6	
공부했어요	5	약 7	
GPT는 한글을 잘게 쪼개요	15	약 18	

영어는 글자 수 ≫ 토큰 수, 한글은 글자 수 ≈ 또는 ≪ 토큰 수. 누가 더 많은 토큰을 쓰는가? _____

2–2 BPE 미니 시뮬레이션 따라하기

초기 어휘는 글자 단위. 가장 자주 붙어 다니는 두 조각을 합쳐가며 토큰을 만들어 갑니다.

예시 코퍼스 "low", "lower", "newest", "widest" (각각 5번씩)

아래 단계에서 다음에 합칠 두 조각을 적어보세요. (앱 시뮬레이터의 진행 순서를 따라가며)

단계	현재 빈도 1위 페어	합친 결과
1		
2		
3		

2–3 한글이 토큰을 더 많이 차지하는 이유

"공부했어요"가 1글자처럼 보이지만 토큰이 5~8개로 쪼개지는 이유를 학습 코퍼스 관점에서 설명하세요.

> **실용 영향** 같은 분량의 글이라도 한글이 영어보다 **토큰 단가**가 더 비싸다 → API 사용료 차이의 원인.

STEP 3. 원-핫에서 임베딩으로

3–1 원-핫의 한계 — 직접 계산

어휘 V = 6 (1번 고양이, 2번 강아지, 3번 호랑이, 4번 사과, 5번 바나나, 6번 컴퓨터)일 때 두 단어의 원-핫 벡터:

```
"고양이" (1번 자리) = ( _____, _____, _____, _____, _____, _____ )
"강아지" (2번 자리) = ( _____, _____, _____, _____, _____, _____ )
내적 = 1·0 + 0·1 + 0·0 + 0·0 + 0·0 + 0·0 = _____
cos(고양이, 강아지) = 내적 / (크기 × 크기) = _____ / (1 × 1) = _____
```

"고양이"와 "강아지"는 사람 보기에 매우 비슷한 단어인데, cos = 0이 나왔다. 같은 어휘 안의 다른 단어 쌍(예: 고양이-컴퓨터)도 cos = 0. 즉 원-핫에서는 모든 다른 단어 쌍이 똑같이 cos = 0. 이 현상이 왜 문제가 될까요?

3-2 임베딩 결과 — 미리 제공

위 6개 단어를 학습된 3차원 임베딩으로 옮긴 결과를 표로 제공:

단어	z_1	z_2	z_3
고양이	0.8	0.6	0.1
강아지	0.7	0.7	0.1
호랑이	0.9	0.5	0.2
사과	0.1	0.2	0.9
바나나	0.2	0.1	0.8
컴퓨터	−0.5	−0.4	−0.3

표를 보고 "가까이 모인 그룹"을 찾아보세요.

가까이 모인 그룹	의미적 공통점

3-3 임베딩 행렬 W의 정체

단어 → 임베딩 벡터로 가는 과정은 결국 **원-핫 × W**의 곱셈. W의 크기는 V × d (어휘 × 임베딩 차원).

```
어휘 V = 6, 임베딩 차원 d = 3일 때 W의 크기 = _____ × _____ = 학습할 숫자 _____개
```

"원-핫 × W = 임베딩"의 곱셈 결과는 사실상 W의 어떤 부분을 그대로 꺼내오는 것과 같다. 어떤 부분인가?

STEP 4. Word2Vec 미니 — 의미 연산

4–1 의미 산수 결과 — 미리 제공

잘 학습된 Word2Vec 모델이 다음 식을 계산하면 가장 가까운 단어가 다음과 같이 나옵니다.

식	1순위 결과	기대 답	적중?
king – man + woman	queen	queen	✓
prince – man + woman	princess	princess	✓
왕 – 남자 + 여자	여왕	여왕	✓
아빠 – 남자 + 여자	엄마	엄마	✓

위 4문제 모두 정답이 1순위로 나왔다. 한국어·영어가 모두 가능하다는 뜻은 임베딩이 어떤 능력을 가졌다는 것인가? 한 줄로:

4–2 의미 차 = 벡터 차 — 직접 계산

간단한 2차원 임베딩 예시. (실제 학습 결과를 단순화한 값.)

단어	z_1	z_2
king	0.8	0.6
man	0.5	0.4
woman	0.5	0.8
queen	0.8	1.0

vec(king) – vec(man) + vec(woman) 을 직접 계산해 vec(queen)과 비교하세요.

```
z1: 0.8 - 0.5 + 0.5 = __________
z2: 0.6 - 0.4 + 0.8 = __________
결과 ( ______, ______ ) ≈ vec(queen) = (0.8, 1.0). 같은가? __________
```

"king → queen"의 화살표(벡터 차)와 "man → woman"의 화살표를 두 단어 좌표 차로 계산해보세요.

```
queen - king = (0.8-0.8, 1.0-0.6) = ( ______, ______ )
woman - man = (0.5-0.5, 0.8-0.4) = ( ______, ______ )
두 화살표는 (같다 / 다르다). ☞ __________
```

두 화살표가 같다는 사실이 의미하는 바는?

4-3 **실패 사례 — 추측**

의미 산수가 잘 안 통하는 경우도 있다. 예: 학습 코퍼스에 거의 등장하지 않은 단어("호랑이")로 같은 산수를 시도하면 1순위가 다른 단어로 나오기 쉽다. 이유를 한 줄로:

STEP 5. 6부 전반 마무리

정리 **한 줄 정리 빈칸**

```
글자는 (  ____________  )와 (  ____________  )를 거쳐 0과 1이 되고,
문장은 (  ____________  ) 단위로 쪼개진 뒤 (  ____________  ) 벡터로 변환된다.
의미가 비슷한 단어가 (  ____________  ) 곳에 모이도록 학습된 결과,
벡터의 (  ____________  )과 (  ____________  )이 의미의 연산이 된다.
```

서술 **친구에게 설명하기**

"GPT는 글자가 아니라 토큰을 본다. 그리고 각 토큰을 임베딩 벡터로 바꾼다." 이 두 문장을 6부 전반에서 배운 용어를 사용해 5~7문장으로 풀어 설명하세요.